

LAB Semana 6 — AR básica

Cómo funciona, de punta a punta, y las partes importantes de cada script

Grupo 7 · Métodos Computacionales en Obras Civiles, UAndes · Pedro Castillo, Monserrat Cubillos, Eduardo Vergara

Qué hace. Una app de realidad aumentada que corre en el **Safari del iPhone**. Reconoce una imagen impresa, la de la derecha, y dibuja encima la **columna 200037** del edificio (el **conjunto**, los dos cuerpos en un solo modelo) y su sector: 21 barras con **el mismo elementTag de OpenSees**. Al lado muestra sus **resultados de OpenSees**.

La regla de oro. Python/OpenSees calcula, el JSON transporta y el teléfono dibuja. En el iPhone no se resuelve nada: solo se transforman coordenadas y se pinta.

Por qué web y no Unity. Los tres tenemos iPhone y no hay Mac. La AR de Unity en iOS (AR Foundation con ARKit) exige compilar en Xcode, que solo existe en macOS. En el navegador, **MindAR** hace el seguimiento de la imagen y **three.js** dibuja.

Dos modos. *Sitio:* la imagen va pegada en la cara +x de la columna real, a 1.40 m, y el modelo se dibuja a tamaño real (1:1). *Maqueta:* la imagen va sobre una mesa, y el sector se dibuja a escala 1:50.

La cadena, en seis pasos

marker

la imagen impresa

pose

MindAR: imagen
→ cámara

anchor

sistema de la
imagen

transform

$M = S(k) \cdot R^T \cdot T(-c)$

elemento

200037, con su tag

resultado

de OpenSees, en
ar.json

Lo que se ve en el teléfono (caso

1.2G+1.0Q+1.4EX):

`elementTag 200037 · columna 1t2:P 0.70x0.70`
 $P = 3701.8 \text{ kN} \cdot M = 507.6 \text{ kN}\cdot\text{m} \cdot M_n(P) = 1721.9 \text{ kN}\cdot\text{m} \cdot u = 0.295$ **PASA**

El caso que gobierna entre las 10 combinaciones mayoradas es 0.9G+1.4EX, con $u = 0.301$.

Dónde está cada cosa

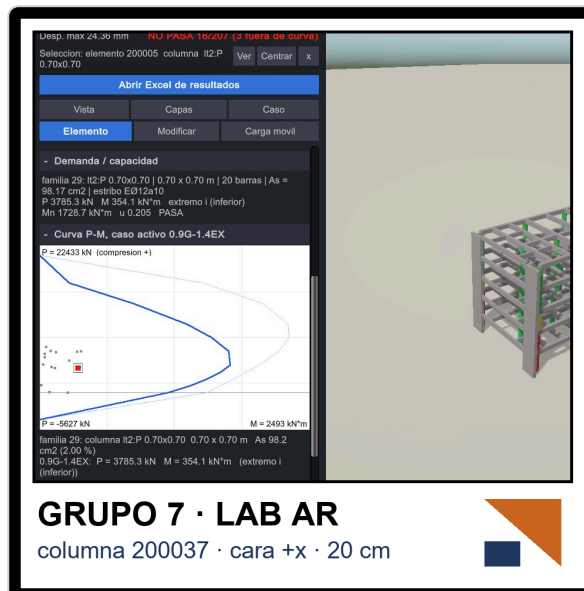
`semana06_lab/`: la app y sus scripts

`semana06_lab/web/`: lo que corre en el iPhone

`semana06_lab/README.md`, `COORDENADAS.md`,

`GUION_DEMO.md`

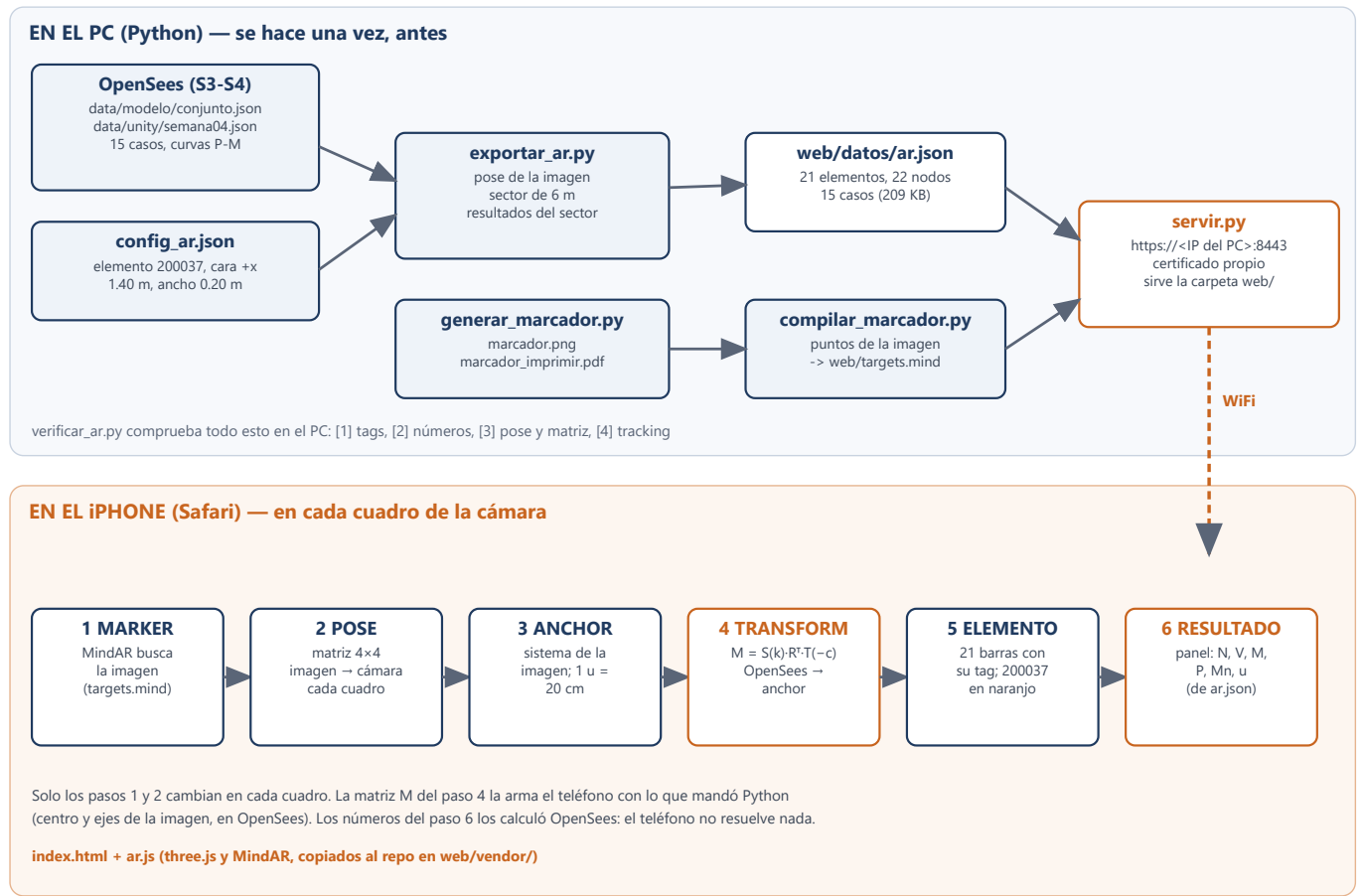
`reports/semana06.md`: el informe del avance



La imagen de referencia (`marcador/marcador_imprimir.pdf`, se imprime a 20.0 cm). Sirve también mostrada en la pantalla del PC.

1. El flujo completo

Hay dos momentos. **En el PC**, una vez y antes de la demo, Python prepara todo lo que el teléfono necesita y lo deja en la carpeta `web/`. **En el iPhone**, en cada cuadro de la cámara, la app busca la imagen, estima su pose y dibuja el modelo encima.



Qué viaja en `web/datos/ar.json` (209 KB)

clave	qué es
marcaador	la pose de la imagen en el edificio: centro y ejes en coordenadas de OpenSees, y el ancho impreso; la versión de maqueta va aparte
nodos, elementos	22 nodos y 21 elementos del sector (columna 1, viga_x 8, viga_y 12), con los mismos tags de OpenSees, la sección y los ejes locales
casos	15 casos (G, Q, EX, EY, S3 y las 10 combinaciones mayoradas): desplazamientos, esfuerzos por estación y la demanda P-M
familias	la curva P-M de cada sección con fierro (la de la 200037 es la familia 29: 20 Ø25, E Ø12a10)
info	de dónde salen los resultados, las unidades, el mapeo al anchor, la escala de la deformada (x160) y el caso por defecto

2. Cómo se corre

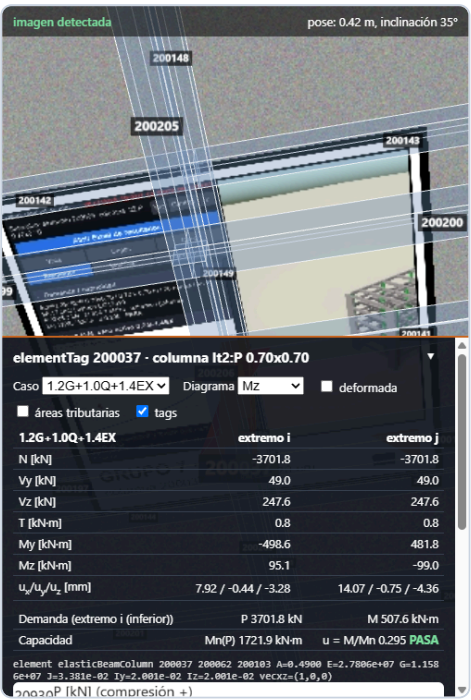
En el PC

```
.\venv\Scripts\python.exe semana06_lab\exportar_ar.py # solo si cambia config_ar.json
.\venv\Scripts\python.exe semana06_lab\servir.py # deja la terminal abierta
.\venv\Scripts\python.exe semana06_lab\verificar_ar.py # los 4 bloques, ~30 s
```

servir.py imprime la dirección, por ejemplo `https://192.168.1.13:8443`. La primera vez Windows pregunta si Python puede recibir conexiones: hay que permitirlo en **redes privadas**.

En el iPhone (mismo WiFi)

1. Abrir esa dirección en **Safari**.
2. Safari avisa "Esta conexión no es privada": **Mostrar detalles** → **visitar este sitio web**. Puede salir un segundo aviso, que se acepta igual. El certificado lo firmó el PC, no una autoridad.
3. Elegir **Maqueta** (o **En sitio**) y **permitir la cámara**.
4. Apuntar a la imagen: impresa al 100 %, o abierta grande en la pantalla del PC. Que ocupe buena parte de la pantalla del teléfono, a 30–50 cm, sin reflejos.
5. Arriba aparece **"imagen detectada"** y la columna en naranja; abajo, el panel. Tocar una barra la selecciona.



La app en modo maqueta. Esta captura es de **Chrome con la cámara sintética de verificar_ar.py**, no de un iPhone.

Si algo sale mal

síntoma	qué hacer
Safari no abre	mismo WiFi que el PC; https y puerto 8443 ; Windows tiene que dejar entrar a Python
el segundo aviso no avanza	esperar unos segundos (la primera vez descarga unos 4 MB: MindAR, three.js y el modelo); si no carga, recargar la página
no pide la cámara	Ajustes → Safari → Cámara → Permitir; recargar
no detecta la imagen	más luz, sin reflejos; que la imagen ocupe al menos un tercio de la pantalla
modelo de tamaño raro	la imagen no mide 20 cm (por ejemplo, en la pantalla del PC): es esperable; para medir, imprimirla
el modelo tiembla	acercarse; que la imagen esté plana

3. Las coordenadas y la matriz

Hay cuatro sistemas: **OpenSees** (x este, y norte, z arriba, en metros), la **imagen** (origen en su centro, x a la derecha, y hacia arriba de la imagen, z saliendo de ella, en metros), el **anchor** de MindAR (los mismos ejes, pero 1 unidad = el ancho impreso, 0.2 m) y la **cámara**, que se mueve.

La pose de la imagen **dentro del edificio** es un dato del modelo: Python la calcula con `pose_del_marcador`. Con su centro **c** y sus ejes **R** = [e_x e_y e_z], todo en OpenSees, un punto **p** del modelo pasa a la imagen y al anchor así:

$$q = R^T (p - c) \quad a = k \cdot q, \quad k = \text{escala} / \text{ancho}$$

$$M = S(k) \cdot R^T \cdot T(-c) \rightarrow x_{\text{pantalla}} = P_{\text{MindAR}} \cdot T_{\text{cámara} \leftarrow \text{anchor}}(t) \cdot M \cdot p$$

La matriz **M** es **fija**, porque no cambia mientras no se mueva la imagen. La pose **T(t)** la entrega MindAR en cada cuadro. **M** es la composición de una traslación, una rotación y una escala uniforme, con determinante $k^3 > 0$: no refleja nada. `three.js` y `OpenSees` son de mano derecha. `Unity` es de mano izquierda, y por eso el visor de escritorio sí intercambia y por **z**.

Sitio (escala 1, $k = 5$)

$c = (-2.38, 55.0833, 1.35)$

$e_x = (0, 1, 0)$, $e_y = (0, 0, 1)$, $e_z = (1, 0, 0)$

$$\begin{vmatrix} 0 & 5 & 0 & -275.4165 \\ 0 & 0 & 5 & -6.75 \\ 5 & 0 & 0 & 11.9 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Techo de la columna, nodo 200103: $p = (-2.73, 55.0833, 3.91) \rightarrow q = (0, 2.56, -0.35) \text{ m} \rightarrow a = (0, 12.8, -1.75)$ anchos. Está 2.56 m sobre el centro de la imagen y 0.35 m detrás de ella (el eje de la columna).

Maqueta (escala 1:50, $k = 0.1$)

$c = (-2.73, 55.0833, -0.05)$ (la base de la columna)

$e_x = (0, 1, 0)$, $e_y = (-1, 0, 0)$, $e_z = (0, 0, 1)$

$$\begin{vmatrix} 0 & 0.1 & 0 & -5.5083 \\ -0.1 & 0 & 0 & -0.273 \\ 0 & 0 & 0.1 & 0.005 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

El mismo nodo queda en $a = (0, 0, 0.396)$ anchos, o sea a 7.92 cm sobre la mesa: la columna de 3.96 m se dibuja de 7.9 cm.

Cómo se comprobó (`verificar_ar.py`)

bloque	qué prueba	resultado
[1] IDs	los tags, nodos, tipo y sección de la app son los del modelo y los del anexo	21 elementos y 22 nodos iguales
[2] resultados	cada número de la app es el del anexo de OpenSees	23 024 números idénticos, bit a bit
[3a] pose	ejes ortonormales y derechos; centro en la cara, a 1.40 m	residuo 0.0e+00
[3b] matriz	la matriz de <code>ar.js</code> (en Chrome) contra la de Python, en los 22 nodos	peor 2.8e-14 anchos
[4] tracking	video sintético con pose conocida $\rightarrow$ la pose que estima la app	0.45 m: 1.1 mm, 1.1°, 0.55 m: 0.6 mm, 0.7°, 0.60 m: 2.6 mm, 0.9°

4. Los scripts, uno por uno

config_ar.json — lo único que se cambia para usar otra columna

Rol. los parámetros: qué columna, en qué cara, a qué altura, el ancho impreso y el radio del sector

Lee. —

Escribe. —

Corre en. lo leen `exportar_ar.py` y `generar_marcador.py`

línea	parámetro y valor	qué significa
4	"elemento": 200037	la columna donde se pega la imagen (su elementTag de OpenSees)
7	"cara": "+x"	la cara de la columna: +x, -x, +y o -y, en ejes de OpenSees
10	"altura_centro_m": 1.40	altura del centro de la imagen sobre el nodo inferior (m)
13	"ancho_impreso_m": 0.20	el ancho REAL de la imagen impresa (m): MindAR mide todo en anchos de imagen
16	"radio_sector_m": 6.0	qué barras se muestran alrededor de la columna (m)
20	"sitio": {"escala": 1.0, ...}	modo sitio: escala 1 (tamaño real)
21	"maqueta": {"escala": 0.02, ...}	modo maqueta: escala 0.02 (1:50)
25	"caso_por_defecto": "1.2G+1.0Q+1.4EX"	el caso con que abre el panel

exportar_ar.py — el último paso de Python

Rol. arma `ar.json`: la pose de la imagen, el sector y sus resultados

Lee. `data/modelo/conjunto.json`, el anexo `semana04.json`, las tributarias

Escribe. `web/datos/ar.json`

Corre en. PC

semana06_lab/exportar_ar.py:82 la transformación

```
82 def a_marcador(p, marcador):
83     """Un punto de OpenSees (m) en el sistema de la imagen (m):
84     q = R^T (p - c), con R = [ex ey ez] en columnas."""
85     d = resta(p, marcador['centro'])
86     return [punto(d, marcador['ejes']['x']), punto(d, marcador['ejes']['y']), punto(d, marcador['ejes']['z'])]
87
88
89 def a_anchor(p, marcador, escala):
90     """Lo mismo, en unidades del anchor de MindAR: 1 unidad = el ancho
91     impreso de la imagen. Es lo que hace ar.js (matrizModeloAAncor)."""
92     q = a_marcador(p, marcador)
93     k = escala / marcador['ancho_m']
94     return [q[0] * k, q[1] * k, q[2] * k]
```

La transformación, en Python. `a_marcador` pasa un punto de OpenSees al sistema de la imagen: $q = R^T(p - c)$. `a_anchor` lo escala a "anchos de imagen" ($k = \text{escala}/\text{ancho}$). Es la misma cuenta que hace `ar.js` en el teléfono, y `verificar_ar.py` [3b] compara las dos.

semana06_lab/exportar_ar.py:105 de dónde salen los números

```
105 def cargar_anexo():
106     """El anexo del conjunto. Si el de disco es de otro edificio (la suite
107     deja el LT2), se arma en memoria con el mismo código que lo exporta."""
108     ruta = os.path.join(rutas.UNITY, 'semana04.json')
109     with io.open(ruta, encoding='utf-8') as fh:
110         anexo = json.load(fh)
111     en_disco = anexo['info']['edificio']
112     if en_disco == EDIFICIO:
113         return anexo, os.path.relpath(ruta, rutas.RAIZ)
114     # semana03/ TAMBIEN tiene un exportar_unity.py: semana04/ tiene que ir
115     # primero en la ruta, o se importa ese y no tiene construir_anexo.
116     rutas.entrar(__file__, os.path.join(rutas.RAIZ, 'semana03'))
117     s4 = os.path.join(rutas.RAIZ, 'semana04')
118     if s4 in sys.path:
119         sys.path.remove(s4)
120     sys.path.insert(0, s4)
121     sys.modules.pop('exportar_unity', None)
122     import exportar_unity as eu # noqa: E402
123     with open(os.devnull, 'w') as dn, contextlib.redirect_stdout(dn):
124         anexo, _ = eu.construir_anexo(EDIFICIO)
125     return anexo, 'semana04/exportar_unity.construir_anexo(%r) en memoria (el de disco era %s)' % (
126         EDIFICIO, en_disco)
```

Los resultados vienen de OpenSees. Lee el anexo de la Semana 4 (`data/unity/semana04.json`). Si la suite dejó en disco el del LT2, arma el del conjunto en memoria con el mismo código que lo exporta.

semana06_lab/exportar_ar.py:146 pose_del_marcaador: dónde queda la imagen

```
146 cara = cfg['cara']
147 normal = list(CARAS[cara])
148 # Medio ancho de la seccion en la direccion de la normal, con la misma
149 # convencion con que ar.js dibuja la barra: h a lo largo del eje local
150 # z y b del y (en una viga, h es el alto y z la vertical). En una
151 # columna cuadrada da lo mismo.
152 b, h = float(anexo_elem['b']), float(anexo_elem['h'])
153 local_z = elemento.get('localZ') or [1.0, 0.0, 0.0]
154 medio = (h if abs(punto(normal, local_z)) > 0.5 else b) / 2.0
155 z_base = min(n1['z'], n2['z'])
156 centro = [n1['x'] + normal[0] * medio, n1['y'] + normal[1] * medio, z_base +
float(cfg['altura_centro_m'])]
157 ez = normal # sale de la imagen hacia quien la mira
158 ey = [0.0, 0.0, 1.0] # arriba de la imagen = arriba del edificio
159 ex = cruz(ey, ez) # derecha de la imagen: sistema derecho
160 # MAQUETA: la misma imagen, acostada sobre una mesa. Ahi la normal de
161 # la imagen es la vertical, asi que el arriba del edificio (+z de
162 # OpenSees) tiene que salir de la imagen. El origen es la base de la
163 # columna, para que el sector se pare sobre la imagen.
164 base = [n1['x'], n1['y'], z_base]
165 mz = [0.0, 0.0, 1.0]
166 mx = ex # la derecha de la imagen, igual que en sitio
167 my = cruz(mz, mx)
```

Dónde está la imagen dentro del edificio. Centro = eje de la columna + medio ancho hacia la cara + altura sobre el nodo inferior. Ejes: z = normal de la cara, y = arriba (+z de OpenSees), x = y x z. En maqueta, la imagen va acostada en la base de la columna.

semana06_lab/exportar_ar.py:191 el sector

```
191 def sector(modelo, objetivo, radio):
192     nodos = {int(n['id']): n for n in modelo['nodos']}
193     n1, n2 = nodos[objetivo['n1']], nodos[objetivo['n2']]
194     z_abajo, z_arriba = min(n1['z'], n2['z']), max(n1['z'], n2['z'])
195     cx, cy = n1['x'], n1['y']
196     tol = 1e-3
197     elegidos = []
198     for e in modelo['elementos']:
199         if e['tipo'] in NO_SE_DIBUJAN:
200             continue
201         a, b = nodos[e['n1']], nodos[e['n2']]
202         cerca = any(math.hypot(p['x'] - cx, p['y'] - cy) <= radio + tol for p in (a, b))
203         dentro = (z_abajo - tol <= min(a['z'], b['z'])) and (max(a['z'], b['z']) <= z_arriba + tol)
204         if cerca and dentro:
205             elegidos.append(e)
206     ids_nodos = sorted({e['n1'] for e in elegidos} | {e['n2'] for e in elegidos})
207     return elegidos, ids_nodos
```

Qué barras se muestran. Las que tienen algún extremo a menos de `radio_sector_m` (6 m) del eje de la columna y quedan entre su piso y su techo. Los brazos rígidos de los muros no se dibujan.

semana06_lab/exportar_ar.py:298 lo que se escribe

```
298     'marcador': marcador,
299     'objetivo': obj_id,
300     'nodos': salida_nodos,
301     'elementos': salida_elem,
302     'familias': familias,
303     'tributarias': tributarias,
304     'casos': casos,
305 }
```

Lo que viaja al teléfono, todo en coordenadas y unidades de OpenSees, con los mismos tags.

marcador/generar_marcador.py y compilar_marcador.py — la imagen de referencia

Rol. fabrican la imagen y la "compilan" para MindAR

Lee. una captura del visor y config_ar.json (el rótulo)

Escribe. marcador.png, marcador_imprimir.pdf (20.0 cm, con regla de 10 cm), web/targets.mind

Corre en. PC, una vez

MindAR reconoce la imagen por sus **esquinas**: puntos de contraste fuerte que no se repiten. Se compararon cuatro candidatas compilándolas con MindAR, y ganó la captura del visor del conjunto **sin reescalar** (agrandarla la vuelve borrosa y pierde esquinas). El contenido de la foto no importa para el tracking: lo que dice qué columna es, es el rótulo.

semana06_lab/marcador/generar_marcador.py:11 las candidatas (puntos de detección / de seguimiento)

11	captura del visor, conjunto, sin agrandar	281 / 27 + 11	<- esta
12	captura del visor, LT2	338 / 31 + 10	
13	planta del conjunto dibujada del modelo	234 / 13 + 6	(grilla repetida)
14	vista 3D del modelo con IDs	184 / 9 + 7	
15	(y la primera: una captura AGRANDADA	... / 21 + 12, borrosa)	

compilar_marcador.py abre <web/herramientas/compilar.html> en Chrome sin ventana. Ahí corre el compilador de MindAR, que es JavaScript, y deja los puntos de la imagen en <web/targets.mind>.

servir.py — llevar la app al teléfono

Rol. sirve la carpeta web/ por https a la red local

Lee. web/

Escribe. el certificado, en .cert/ (no se versiona)

Corre en. PC, durante la demo

semana06_lab/servir.py:134 https en 0.0.0.0:8443

```
134     puerto = args.puerto or 8443
135     ips = ips_locales()
136     crt, key = certificado(ips)
137     srv = http.server.ThreadingHTTPServer(('0.0.0.0', puerto), manejador)
138     ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
139     ctx.load_cert_chain(crt, key)
140     srv.socket = ctx.wrap_socket(srv.socket, server_side=True)
141     print('=' * 64)
```

semana06_lab/servir.py:74 el certificado

```
74 def certificado(ips):
75     os.makedirs(CERT, exist_ok=True)
76     crt, key = os.path.join(CERT, 'cert.pem'), os.path.join(CERT, 'key.pem')
77     marca = os.path.join(CERT, 'ips.txt')
78     san = ','.join(['DNS:localhost', 'IP:127.0.0.1'] + ['IP:%s' % i for i in ips])
79     # Se rehace si cambio la IP del PC o si le queda poco: dura 60 días.
80     if (os.path.exists(crt) and os.path.exists(marca) and open(marca).read() == san
81         and time.time() - os.path.getmtime(crt) < 50 * 86400):
82         return crt, key
83     openssl = buscar_openssl()
84     if not openssl:
85         raise SystemExit('No encuentro openssl (viene con Git for Windows). Instala Git o usa --http.')
86     subprocess.run([openssl, 'req', '-x509', '-newkey', 'rsa:2048', '-nodes', '-days', '60',
87                     '-keyout', key, '-out', crt, '-subj', '/CN=Grupo 7 LAB AR',
88                     '-addext', 'subjectAltName=' + san], check=True,
89                     stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
90     with open(marca, 'w') as fh:
91         fh.write(san)
92     return crt, key
```

Safari exige https para abrir la cámara. Se genera un certificado firmado por el PC para sus IPs (con el openssl de Git). Se rehace si cambia la IP o si le quedan menos de 10 días. Por eso Safari avisa "Esta conexión no es privada".

semana06_lab/servir.py:100

```
100     def end_headers(self):
101         # Sin cache: si se reexporta ar.json, el telefono ve lo nuevo.
102         self.send_header('Cache-Control', 'no-store')
103         super().end_headers()
104
105     def do_POST(self):
106         partes = urllib.parse.urlparse(self.path)
107         nombre = urllib.parse.parse_qs(partes.query).get('nombre', [''])[0]
108         if partes.path != '/guardar' or nombre not in PERMITIDOS or self.client_address[0] != '127.0.0.1':
109             self.send_error(403)
110             return
111         largo = int(self.headers.get('Content-Length', 0))
112         datos = self.rfile.read(largo)
113         with open(os.path.join(WEB, nombre), 'wb') as fh:
114             fh.write(datos)
115         print(' guardado web/%s (%d bytes)' % (nombre, len(datos)))
116         self.send_response(200)
117         self.end_headers()
118         self.wfile.write(b'ok')
```

Sin caché, para que el teléfono vea un `ar.json` recién exportado. El POST solo lo acepta desde el propio PC (lo usa `compilar_marcaador.py` para guardar `targets.mind`).

web/ar.js — la app (corre en el teléfono)

Rol. sesión AR, tracking, pose, anchor, transformación, dibujo y panel

Lee. datos/ar.json, targets.mind

Escribe. nada

Corre en. **iPhone** (Safari)

semana06_lab/web/ar.js:34 1. leer lo que mandó Python

```
34 const D = await (await fetch('datos/ar.json', { cache: 'no-store' })).json();
35 const NODOS = new Map(D.nodos.map(n => [n.id, n]));
36 const ELEM = new Map(D.elementos.map(e => [e.id, e]));
37 const CASOS = new Map(D.casos.map(c => [c.nombre, c]));
```

Lo primero que hace la app: leer `ar.json`. Ningún número estructural se calcula después.

semana06_lab/web/ar.js:44 2. la matriz M

```
44 export function matrizModeloAAAnchor(pose, escala, anchoM) {
45   const k = escala / anchoM;
46   const { x: ex, y: ey, z: ez } = pose.ejes;
47   const c = pose.centro;
48   const dot = (a, b) => a[0] * b[0] + a[1] * b[1] + a[2] * b[2];
49   return new THREE.Matrix4().set(
50     k * ex[0], k * ex[1], k * ex[2], -k * dot(ex, c),
51     k * ey[0], k * ey[1], k * ey[2], -k * dot(ey, c),
52     k * ez[0], k * ez[1], k * ez[2], -k * dot(ez, c),
53     0, 0, 0, 1);
54 }
```

La matriz $M = S(k) \cdot R^T \cdot T(-c)$. Cada fila es un eje de la imagen, escrito en OpenSees y multiplicado por k, y la última columna es $-k \cdot (\text{eje} \cdot c)$. Se aplica al grupo `modelo` como su matriz.

semana06_lab/web/ar.js:104 3. las barras, con su tag

```
104 function construirBarras() {
105   for (const e of D.elementos) {
106     const { a, b, X, Y, Z, L } = ejes(e);
107     // Caja de L x b x h: h a lo largo del eje local z (en una viga, el
108     // alto), b del y. La misma convencion que usa exportar_ar.py.
109     const geo = new THREE.BoxGeometry(L, e.b || 0.3, e.h || 0.3);
110     const mat = new THREE.MeshBasicMaterial({ color: colorDe(e), transparent: true,
111       opacity: e.id === D.objetivo ? 0.75 : 0.35, depthWrite: false });
112     const malla = new THREE.Mesh(geo, mat);
113     malla.matrixAutoUpdate = false;
114     malla.matrix.makeBasis(X, Y, Z).setPosition(new THREE.Vector3().addVectors(a, b).multiplyScalar(0.5));
115     malla.userData.id = e.id;
116     const bordes = new THREE.LineSegments(new THREE.EdgesGeometry(geo),
117       new THREE.LineBasicMaterial({ color: e.id === D.objetivo ? 0xffb070 : 0xffffffff }));
118     malla.add(bordes);
119     capas.barras.add(malla);
120     mallas.push(malla);
121   }
122 }
```

Cada barra es una caja de $L \times b \times h$, puesta con sus ejes locales (los mismos de OpenSees), y lleva en `userData.id` su `elementTag`.

semana06_lab/web/ar.js:351 4. la sesión AR y el anchor

```
351 async function iniciar(modo) {
352   estado.modo = modo;
353   $('inicio').hidden = true;
354   $('panel').hidden = false;
355   aplicarModo();
356   actualizarPanel();
357   const mindar = new MindARThree({
358     container: $('ar'), imageTargetSrc: 'targets.mind', maxTrack: 1,
359     uiLoading: 'yes', uiScanning: 'no', uiError: 'yes',
360     filterMinCF: 0.0001, filterBeta: 0.001, // suaviza la pose sin retrasarla mucho
361   });
362   const { renderer, scene, camera } = mindar;
363   const anchor = mindar.addAnchor(0); // el ANCHOR: el sistema de la imagen
364   anchor.group.add(modelo);
365   anchor.onTargetFound = () => { $('estado').textContent = 'imagen detectada'; $('estado').className = 'ok';
366   };
367   anchor.onTargetLost = () => { $('estado').textContent = 'buscando la imagen...'; $('estado').className = '';
368   $('pose').textContent = ''; };
369 }
```

La sesión AR. MindAR abre la cámara y busca la imagen compilada en `targets.mind`. `addAnchor(0)` crea el ANCHOR, y el modelo se cuelga de él: desde ahí se mueve pegado a la imagen.

semana06_lab/web/ar.js:369 5. tocar para seleccionar

```
369 const ray = new THREE.Raycaster(), puntero = new THREE.Vector2();
370 $('ar').addEventListener('pointerup', (ev) => {
371   const r = renderer.domElement.getBoundingClientRect();
372   puntero.set(((ev.clientX - r.left) / r.width) * 2 - 1, -((ev.clientY - r.top) / r.height) * 2 + 1);
373   ray.setFromCamera(puntero, camera);
374   const hit = ray.intersectObjects(mallas, false)[0];
375   if (hit) { estado.seleccion = hit.object.userData.id; redibujar(); }
376 });
```

Tocar la pantalla lanza un rayo desde la cámara; la barra que toca queda seleccionada por su `elementTag`.

semana06_lab/web/ar.js:380 6. la pose, cuadro a cuadro

```
380 renderer.setAnimationLoop((t) => {
381   if (anchor.group.visible) {
382     // La pose: la matriz del anchor respecto de la camara, en pixeles de la
383     // imagen (su escala es el ancho en pixeles). Pasada a metros:
384     const m = anchor.group.matrix, e = m.elements;
385     const anchoPx = new THREE.Vector3(e[0], e[1], e[2]).length();
386     const t3 = new THREE.Vector3(e[12], e[13], e[14]);
387     const dist = t3.length() / anchoPx * D.marcador.ancho_m;
388     const normal = new THREE.Vector3(e[8], e[9], e[10]).normalize();
389     const giro = THREE.MathUtils.radToDeg(Math.acos(Math.min(1, Math.abs(normal.z))));
390     $('pose').textContent = `pose: ${dist.toFixed(2)} m, inclinación ${giro.toFixed(0)}°`;
391     if (PRUEBA && t - ultimo > 500) { ultimo = t; log('AR_POSE', { m: Array.from(e), dist_m: dist,
392     giro_deg: giro }); }
393   }
394   renderer.render(scene, camera);
395 }
```

La pose en cada cuadro. MindAR entrega la matriz del anchor en píxeles de la imagen; dividiendo por el ancho en píxeles y multiplicando por el ancho impreso se obtiene la distancia en metros (la barra "pose:").

```

260 function actualizarPanel() {
261   const e = ELEM.get(estado.seleccion);
262   const caso = CASOS.get(estado.caso);
263   const s = caso.esfuerzos[e.id];
264   $('titulo').textContent = `elementTag ${e.id} · ${e.tipo} ${e.seccion}`;
265   const filas = ['N', 'Vy', 'Vz', 'T', 'My', 'Mz'].map(m => {
266     const u = m.startsWith('M') || m === 'T' ? 'kN·m' : 'kN';
267     return `<tr><td>${m} [${u}]</td><td>${f1(s[m][0])}</td><td>${f1(s[m][s[m].length - 1])}</td></tr>`;
268   }).join('');
269   const d1 = caso.desplazamientos[e.n1] || [0, 0, 0], d2 = caso.desplazamientos[e.n2] || [0, 0, 0];
270   const mm = (u) => `${f1(u[0] * 1000, 2)} / ${f1(u[1] * 1000, 2)} / ${f1(u[2] * 1000, 2)}`;
271   let html = `<table><tr><th>${caso.nombre}</th><th>extremo i</th><th>extremo j</th></tr>${filas}
272     <tr><td>u<sub>x</sub></td><td>y</td><td>z</td><td>[mm]</td><td>${mm(d1)}</td><td>${mm(d2)}</td></tr>
</table>`;
273   const dem = caso.demandas[e.id];
274   if (dem) {
275     html += `<table><tr><td>Demanda (extremo ${dem.extremo})</td><td>P ${f1(dem.P)} kN</td><td>M ${f1(dem.M)}
kN·m</td></tr>
276     <tr><td>Capacidad</td><td>Mn(P) ${f1(dem.Mn)} kN·m</td><td>u = M/Mn ${f1(dem.u, 3)}
277     <span class="${dem.pasa ? 'pasa' : 'nopasa'}">${dem.pasa ? 'PASA' : 'NO PASA'}</span></td></tr>
</table>`;
278   }
279   const trib = D.tributarias.find(t => t.elemento === e.id);

```

El panel. Toma los números del caso elegido tal como vienen en `ar.json` y solo los redondea para mostrarlos (`toFixed`).

verificar_ar.py — la app, criterio por criterio, sin teléfono

Rol. los 4 bloques de la rúbrica, con números

Lee. ar.json, el modelo, el anexo, ar.js (en Chrome)

Escribe. nada en el repo (temporales)

Corre en. PC, con Chrome o Edge; en la suite

semana06_lab/verificar_ar.py:83 [1] los tags de la app son los de OpenSees

```
83 def bloque_ids(ar, modelo, anexo, inf):
84     titulo('[1] ELEMENTO / ID: los tags de la app son los de OpenSees')
85     E = {e['id']: e for e in modelo['elementos']}
86     N = {n['id']: n for n in modelo['nodos']}
87     A = {e['id']: e for e in anexo['elementos']}
88     malos = []
89     for e in ar['elementos']:
90         m = E.get(e['id'])
91         if m is None or (m['n1'], m['n2'], m['tipo'], m['seccion']) != (e['n1'], e['n2'], e['tipo'],
e['seccion']):
92             malos.append('elemento %s' % e['id'])
93             continue
94             linea = A[e['id']]['tag_opensees']
```

semana06_lab/verificar_ar.py:243 [4] el video sintético

```
243 def generar_video(ruta, ancho_m, dist, incl_deg, giro_deg, cuadros=45):
244     """La imagen impresa vista desde una cámara con la pose dada, con los
245     intrínsecos de MindAR, escrita como video .y4m (lo que Chrome acepta
246     como cámara falsa). Devuelve la pose verdadera."""
247     import numpy as np
248     from PIL import Image
249     src = np.asarray(Image.open(MARCADOR).convert('RGB'), dtype=np.float32)
250     H0, W0 = src.shape[:2]
251     alto_m = ancho_m * H0 / W0
252     f = (H_VIDEO / 2) / math.tan(FOVY / 2)
253     K = np.array([[f, 0, W_VIDEO / 2], [0, f, H_VIDEO / 2], [0, 0, 1]])
254     # Cámara de frente (convención OpenCV: x derecha, y abajo, z adelante):
255     # x del marcador -> x, y (arriba) -> -y, z (hacia la cámara) -> -z.
256     R0 = [[1, 0, 0], [0, -1, 0], [0, 0, -1]]
257     R = np.array(mat(R0, mat(rot_x(math.radians(incl_deg)), rot_z(math.radians(giro_deg)))))
258     t = np.array([0.0, 0.0, dist])
259     # pixel de la imagen (u, v) -> punto del marcador (X, Y, 0) en metros
260     A = np.array([[ancho_m / W0, 0, -ancho_m / 2], [0, -alto_m / H0, alto_m / 2], [0, 0, 1]])
261     Hm = K @ np.column_stack([R[:, 0], R[:, 1], t]) @ A
262     Hinv = np.linalg.inv(Hm)
```

Una cámara de mentira con pose conocida. Se proyecta la imagen con una homografía $H = K[r_1 \ r_2 \ t]A$ (la cámara de 45° que supone MindAR) y se escribe como video .y4m.

semana06_lab/verificar_ar.py:352 [4] Chrome con cámara falsa

```
352     args += ['--use-fake-ui-for-media-stream', '--use-fake-device-for-media-stream',
353             '--use-file-for-fake-video-capture=' + video]
354     args.append('http://localhost:%d/%s' % (PUERTO, pagina))
```

Chrome sin ventana usa ese video como si fuera la cámara. La app lo ve, detecta la imagen y publica su pose en la consola (AR_POSE), que el script compara con la pose verdadera.

Lo que esto NO prueba. Todo corre en Chrome de escritorio con una cámara ideal: la misma de 45° que supone MindAR. No ve la cámara real del iPhone ni su focal, ni Safari, ni el temblor de la mano, ni la luz. Eso es la prueba en el teléfono, y es lo que falta (última página).

5. Los scripts del avance (semana06/)

Producen los números del informe `reports/semana06.md` y están en la suite.

script	qué hace	resultado de hoy
<code>verificar_semana06.py</code>	la tabla de QA del \$5, en vivo sobre el conjunto, con su criterio escrito en el código (6 s)	8 OK, 2 PARCIAL (P-M muro: el mallado; AR: sin iPhone), 0 FALLA
<code>traza_200037.py</code>	la columna 200037 desde la lámina 2024_22-101 y la elevación 305 hasta el panel del teléfono	app contra OpenSees: 7.65e-05 (cota 2.30e-04)
<code>precision_ar.py</code>	el presupuesto de error del registro, la simulación de la focal y la prueba para el iPhone	sitio 7.3 cm, maqueta 1.7 mm (suma cuadrática, en el techo)

semana06/verificar_semana06.py:186 verificar_semana06: la cota del equilibrio

```
186 def filas_que_cuentan(modelo, reacciones):
187     r"""
188     Cuántas filas de reaccion entran en Fx, Fy y Fz, preguntandose a
189     calcular.equilibrio: con 1 kN en cada fila y ninguna carga, la
190     'reaccion' que devuelve es el conteo. La regla (quien cuenta en cada
191     grado de libertad) sigue teniendo una sola definicion. Es el mismo
192     truco de semana05/verificar_superposicion.filas_que_cuentan.
193     """
194     unos = {'reacciones': [{'id': r['id'], 'fx': 1.0, 'fy': 1.0, 'fz': 1.0}
195                           for r in reacciones]}
196     cuenta = calcular.equilibrio(
197         modelo, {'cargas_nodales': [], 'cargas_distribuidas': []}, unos)
198     return [int(round(v)) for v in cuenta['reaccion_kN']]
```

Cuántas reacciones entran en Fx, Fy y Fz, preguntándose a `calcular.equilibrio` con 1 kN en cada fila: la regla tiene una sola definición. Con eso sale la cota del redondeo.

semana06/precision_ar.py:164 precision_ar: las fuentes de error

```
164 def fuentes(q, modo, trk):
165     r"""
166     Corrimiento de un punto dibujado en q (m, sistema de la imagen) por
167     cada fuente. Las de escala valen eps * |q|: si MindAR estima la
168     distancia un eps de mas, dibuja el modelo como si fuera un eps mas
169     chico alrededor del centro de la imagen (la proyeccion no cambia al
170     escalar todo), y lo mismo pasa si la imagen impresa es un eps mas
171     chica de lo declarado.
172     """
173     r = float(np.linalg.norm(q))
174     angulo = max(float(np.linalg.norm(rot(e, trk['grados']) @ q - q))
175                 for e in ((1, 0, 0), (0, 1, 0), (0, 0, 1)))
176     f = [('tracking, angulo (%.1f grados)' % trk['grados'], angulo),
177         ('tracking, distancia (%.2f %%)' % (100 * trk['escala']), trk['escala'] * r),
178         ('impresion (%.0f %%, supuesto)' % (100 * IMPRESION), IMPRESION * r)]
179     if modo == 'sitio':
180         f += [('colocacion, corrimiento (%.0f mm, supuesto)' % (1000 * COLOCACION_M), COLOCACION_M),
181             ('colocacion, giro (%.0f grado, supuesto)' % COLOCACION_GRADOS,
182              float(np.linalg.norm(rot((0, 0, 1), COLOCACION_GRADOS) @ q - q))]
183     return f
```

Cada fuente de error, **exacta**: se gira o se escala la pose y se mide cuánto se mueve el nodo dibujado.

semana06/traza_200037.py:201 traza_200037: OpenSees sin redondear

```
201     return (list(ops.eleResponse(EID, 'localForce')),
202            [ops.nodeDisp(ni, k) for k in range(1, 7)] + [ops.nodeDisp(nj, k) for k in range(1, 7)])
```

La traza lee las fuerzas de OpenSees **sin** el redondeo del servidor y las compara con `ar.json`, con la cota del redondeo por cada factor λ .

6. Para la defensa

pregunta	respuesta corta
¿Qué corre en el teléfono?	La cámara, el seguimiento de la imagen, la pose, la matriz M y el dibujo. Nada estructural.
¿Qué se calculó antes?	El modelo, los 15 casos en OpenSees, las curvas P-M, la pose de la imagen en el edificio, el sector y <code>targets.mind</code> .
¿Qué es el anchor?	El sistema de coordenadas de la imagen, que MindAR mueve en cada cuadro con la pose. Lo que cuelga de él queda pegado a la imagen.
¿Cómo sabe dónde está la columna?	La imagen se pega en su cara +x a 1.40 m. Python calcula dónde queda eso en el modelo: centro (-2.38, 55.0833, 1.35) y ejes. Con eso arma la matriz M.
¿Es el mismo elemento que en OpenSees?	Sí: el tag del JSON es el de OpenSees (<code>e1eNodes(200037) = [200062, 200103]</code>), y <code>verificar_ar[1]</code> lo comprueba para las 21 barras.
¿Qué error tiene?	Pose medida en video sintético: 0.6–2.6 mm y hasta 1.1°. En sitio da unos 7 cm en el techo de la columna, más 5–10 cm por el nivel de la losa. En maqueta, unos 2 mm. La focal del iPhone no está medida.
¿Qué columna física es?	La del eje C del LT2, de -0.05 a 3.91 m: la tercera de su eje, del piso 2 según las láminas. Hay que confirmarlo en obra.
¿Por qué la $u = 0.295$ si gobierna 0.301?	La app abre en 1.2G+1.0Q+1.4EX ($u = 0.295$). La que gobierna es 0.9G+1.4EX ($u = 0.301$): con menos axial la columna pierde más Mn. Las dos pasan.

Lo que falta, dicho de frente

- **La prueba en un iPhone real.** Con una captura del teléfono en `semana06/evidencia/iphone/`, la fila AR de la tabla de QA pasa sola a OK.
- **Medir el error de verdad:** la cinta a 30, 50 y 80 cm (la barra "pose:" tiene que decir lo mismo) y una regla vertical en maqueta (el techo de la columna a 7.92 cm de la mesa). Se anota en `semana06/evidencia/iphone/LEEME.md`.
- **En sitio:** confirmar la columna y medir el desnivel entre la base dibujada y el piso real.
- La deformada en AR va recta entre nodos; el visor de Windows la dibuja curva.

Generado el 01-10-2026 09:02 a partir del código del repo. Cada fragmento se buscó en su archivo al generar, así que las líneas citadas son las de hoy.